CHAPTER 12:
OVERVIEW OF SOFTWARE ARCHITECTURE

CONTENT

- Software Architecture and Component-based Software Architecture
- Multiple Views of a Software Architecture
- Software Architectural Pattern
- Documenting Software Architectural Patterns
- Interface Design
- Designing Software Architecture

SOFTWARE ARCHITECTURE AND COMPONENT-BASED SOFTWARE ARCHITECTURE

- Definition of software architecture:

  "The software architecture of a program or computing system is the structure or structures of the system, which comprise **software elements**, the **externally visible properties of those elements**, and the **relationships** among them." (Bass, Clements, and Kazman, 2003).

- Primarily considered from a **structural perspective**, but must also be examined from:
  - **Static** and **dynamic** perspectives
  - **Functional** (what the system does) and **nonfunctional** (quality attributes like performance, security) viewpoints.

COMPONENT-BASED SOFTWARE ARCHITECTURE

- Focuses on a **modular structure** of self-contained components.
- Each component:
  - Is **encapsulated** (hides implementation)
  - Has a **defined interface** for communication with other components
  - Can be a **simple** or **composite object**
  - Acts as a **black box** to other components
- Components communicate using **predefined patterns**.
- Components can be compiled separately and reused via libraries.

COMPONENT-BASED SOFTWARE ARCHITECTURE

- In **sequential** design:
  - Components = passive classes (no threads)
  - Uses **call/return** pattern
- **In concurrent / distributed design:**
  - Components are **active** and can be **deployed on different nodes**
  - Supports **synchronous, asynchronous, brokered, group communication**
  - **Middleware** often used to manage component interaction.

ARCHITECTURE STEREOTYPES (UML 2)

- Elements can have **multiple stereotypes**:
  - One for **role** (e.g., boundary, entity)
  - Another for **architecture** (e.g., subsystem, component, service)
- These stereotypes are **orthogonal** (independent), supporting flexible modeling in COMET method

CONTENT

- Software Architecture and Component-based Software Architecture
- Multiple Views of a Software Architecture
- Software Architectural Pattern
- Documenting Software Architectural Patterns
- Interface Design
- Designing Software Architecture

MULTIPLE VIEWS OF A SOFTWARE ARCHITECTURE

The design of the software architecture can be depicted from different perspectives, referred to as **different views**:

- The **structural view**: class diagrams
- The **dynamic view**: communication diagrams
- The **deployment view** : deployment diagrams
- Another view: **component-based software architecture** view

STRUCTURAL VIEW

- A **static view** of the system — does not change over time.
- Depicted using **UML class diagrams**.
- Subsystems modelled as **composite/aggregate classes** with associations.
- Multiplicity shows how many instances of each subsystem exist.

Example: Banking System (Client/Server architecture)

- Multiple **ATM Client** subsystems connect to a single **Banking Service**.
- Association: *ATM Client Requests Service From Banking Service*.
- Multiplicity: **1-to-many** (one service, many clients).
- Stereotypes:
  - **Role**: client, service
  - **Structure**: subsystem

STRUCTURAL VIEW

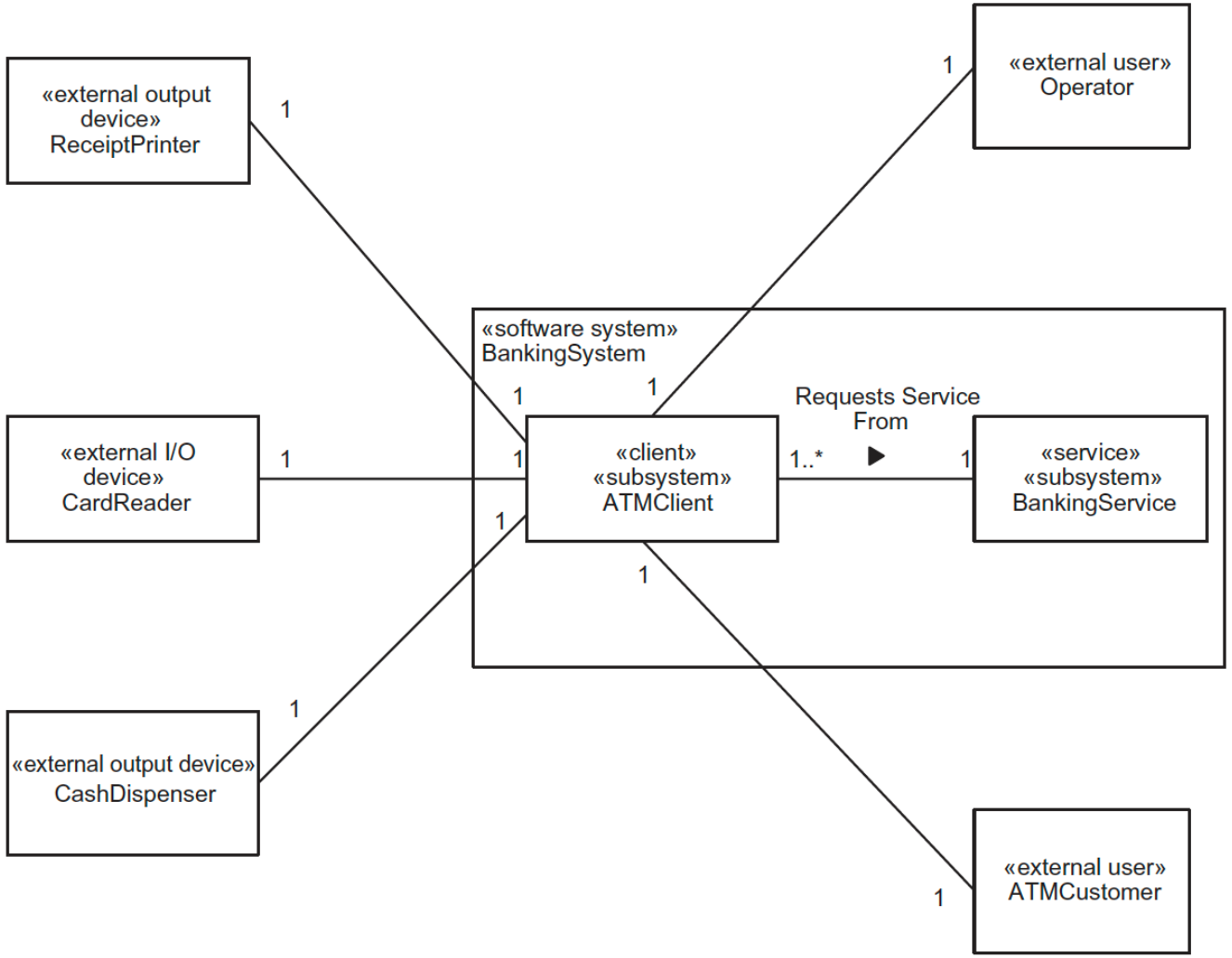


Figure 12.1. Structural view of client/server software architecture: high-level class diagram for Banking System

DYNAMIC VIEW

- A **behavioral view** — focuses on message interactions => Depicted using **UML communication diagrams**
- Shows subsystems as **concurrent components** (executing in parallel, communicate with each other over a network)
- **Example**: Banking System (same scenario as structural view)
  - ATM Clients send transactions to, and receive responses from, Banking Service
  - Communication is **synchronous with reply**:
    - ATMTransaction (solid black arrowhead)
    - bankResponse (dashed arrow with stick arrowhead)
- Diagrams depict **generic interactions** (not instance-specific)
  - No message sequence numbers
  - Object names are **not underlined** (UML 2 convention)
- Highlights **geographically distributed, concurrent subsystems**

DYNAMIC VIEW

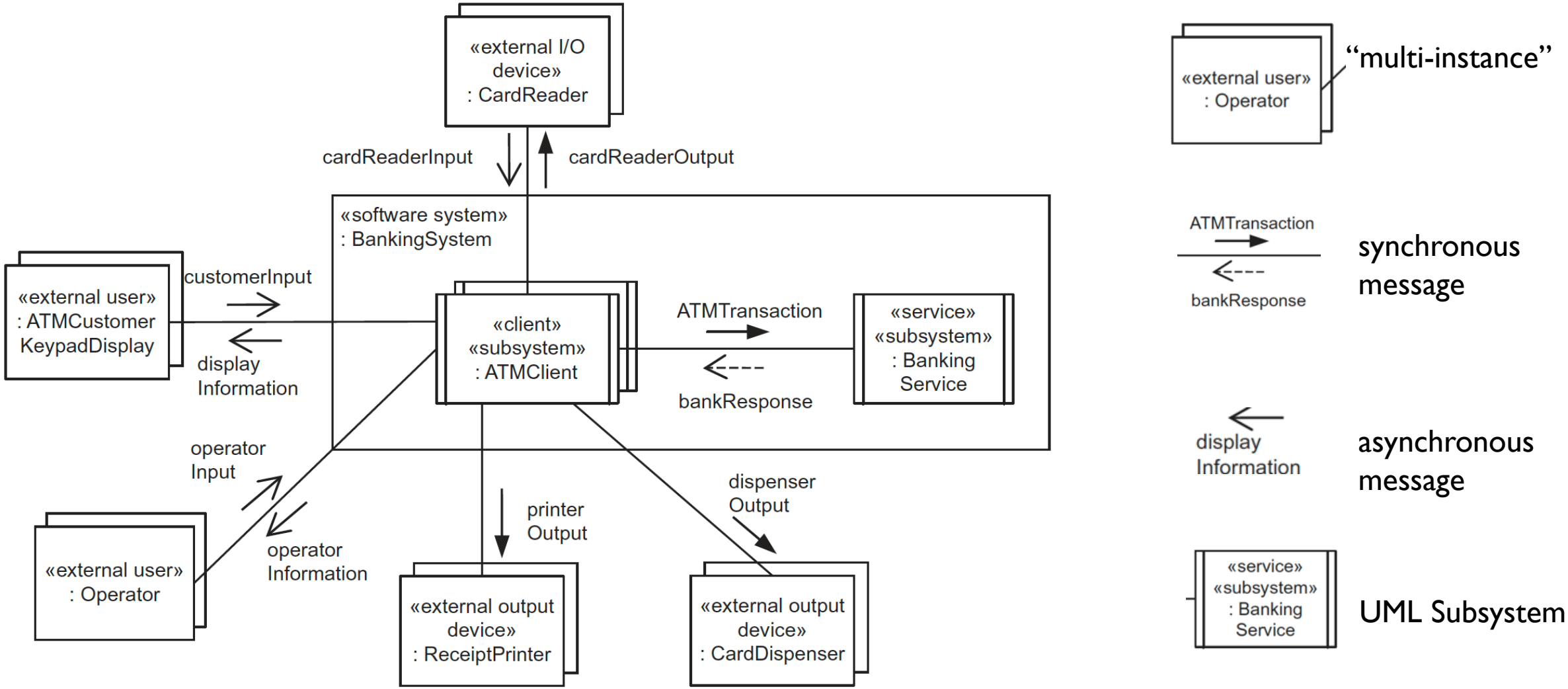
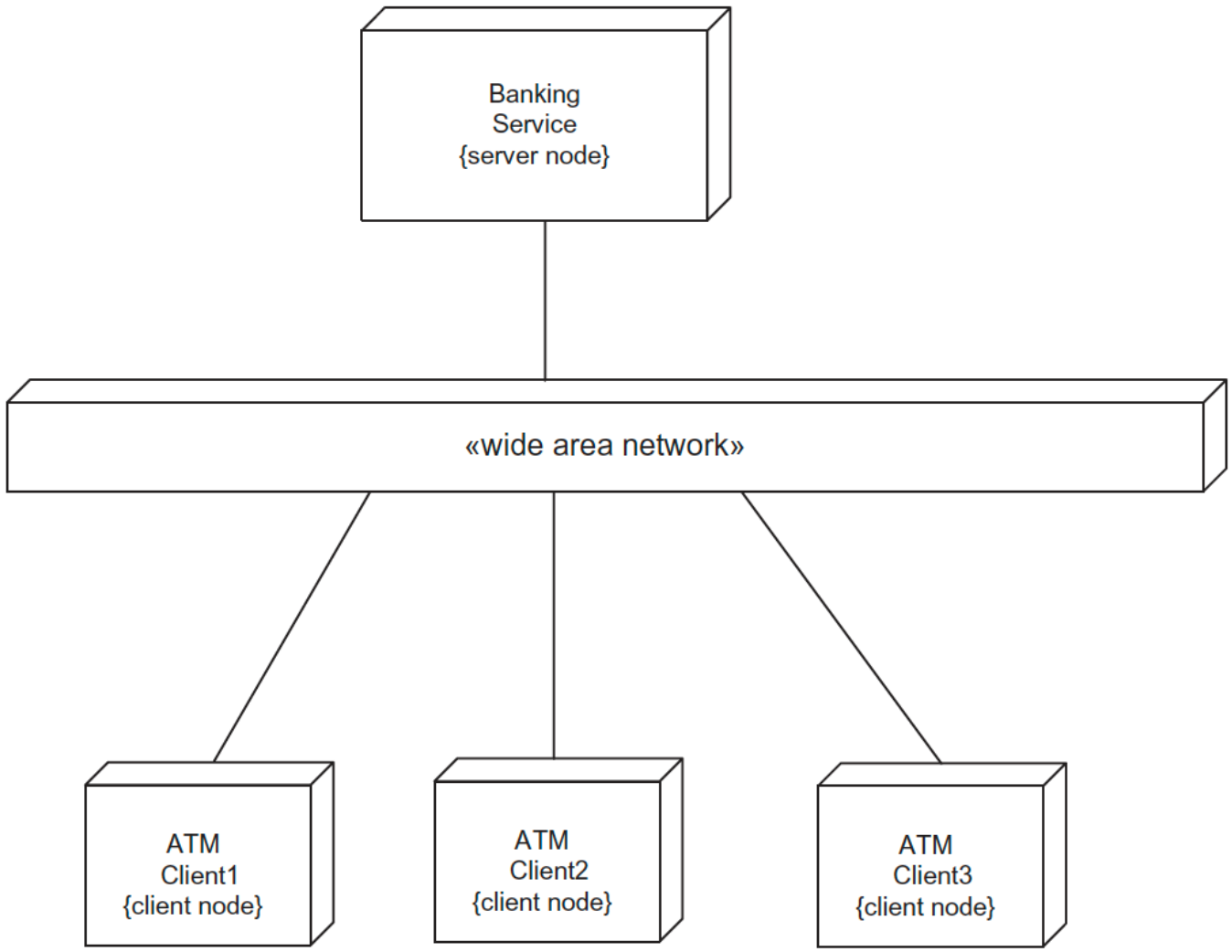


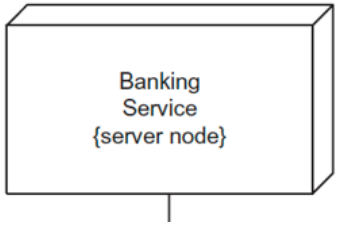
Figure 12.2. Dynamic view of client/server software architecture: high-level communication diagram for Banking System

DEPLOYMENT VIEW

- Shows the **physical configuration** of software architecture
- Uses **UML deployment diagrams** to map subsystems to hardware nodes
- Two styles:
  - **Specific deployment**: exact number of nodes shown
  - **Generic structure**: shows scalability (e.g., many ATM Clients)
- **Example**: Banking System
  - Each **ATM Client** is deployed on a **separate node**
  - A single **Banking Service** is deployed on a **centralized node**
  - Nodes connected via a **Wide Area Network (WAN)**

DEPLOYMENT VIEW





Node

Figure 12.1. Deployment view of client/server software architecture: deployment diagram

CONTENT

- Software Architecture and Component-based Software Architecture
- Multiple Views of a Software Architecture
- Software Architectural Pattern
- Documenting Software Architectural Patterns
- Interface Design
- Designing Software Architecture

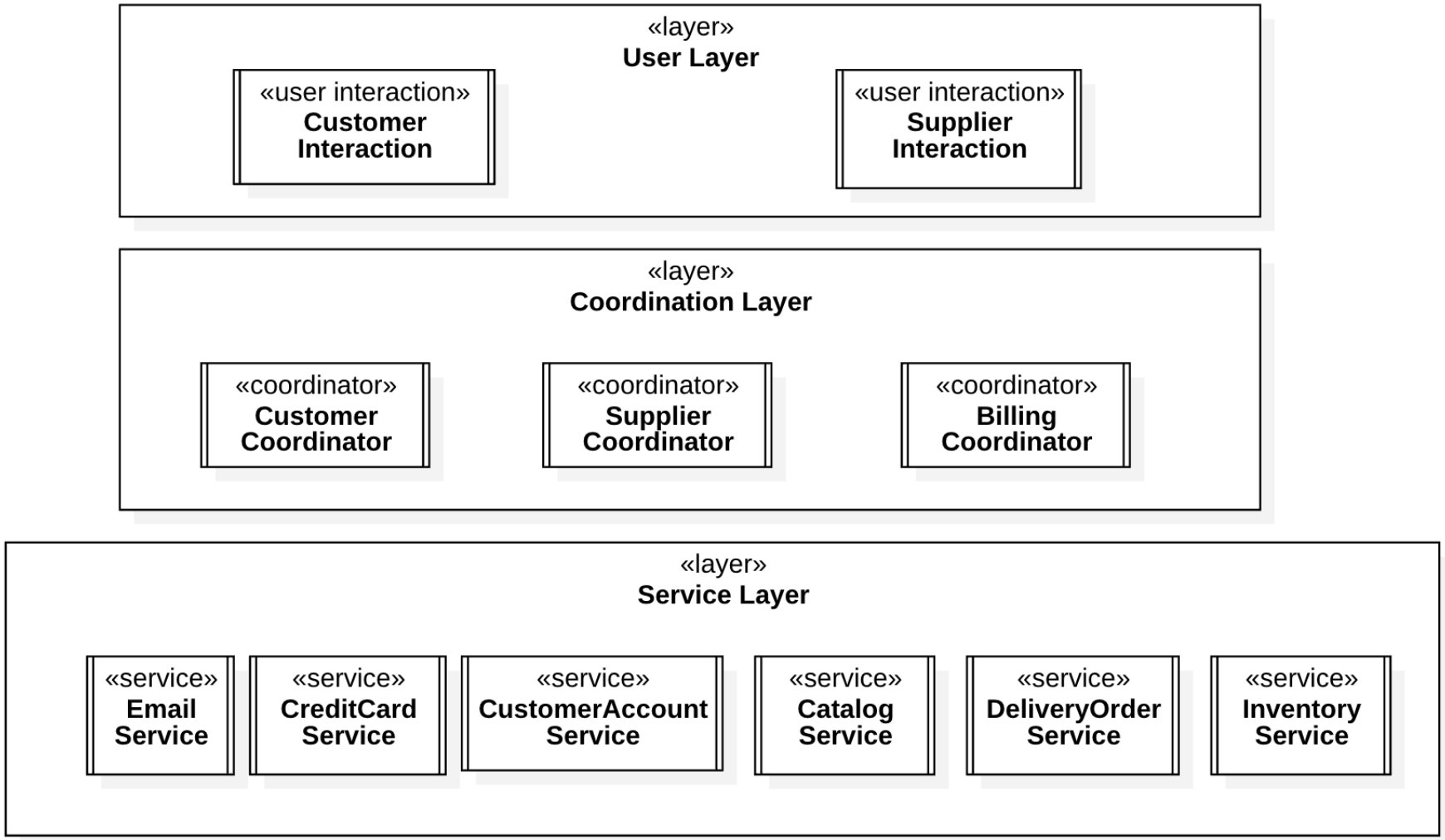
SOFTWARE ARCHITECTURAL PATTERNS

- Architectural patterns = **template/skeleton** for high-level software architecture
- **Two main categories**:
  - **Architectural structure patterns**: address static structure
  - **Architectural communication patterns**: address dynamic communication
- Common architectures: **Client/Server**, **Layered Architecture**

LAYERS OF ABSTRACTION ARCHITECTURAL PATTERN

- Also called **Hierarchical Layers** or **Levels of Abstraction**
- Common in operating systems, databases, network software
- **Strict hierarchy**: Layer only uses immediate lower layer services
- **Flexible hierarchy**: Layer can use services multiple levels below
- Example: TCP/IP 5 Layers:
  - Layer 1: Physical layer
  - Layer 2: Network interface layer
  - Layer 3: Internet layer (IP)
  - Layer 4: Transport layer (TCP)
  - Layer 5: Application layer (e.g., FTP, WWW)
- Easy to **extend** (add layers) or **contract** (remove layers)

LAYERS OF ABSTRACTION ARCHITECTURAL PATTERN



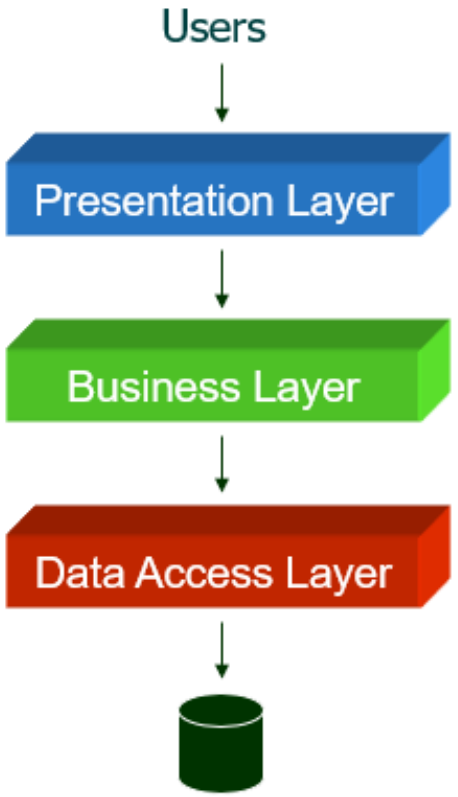
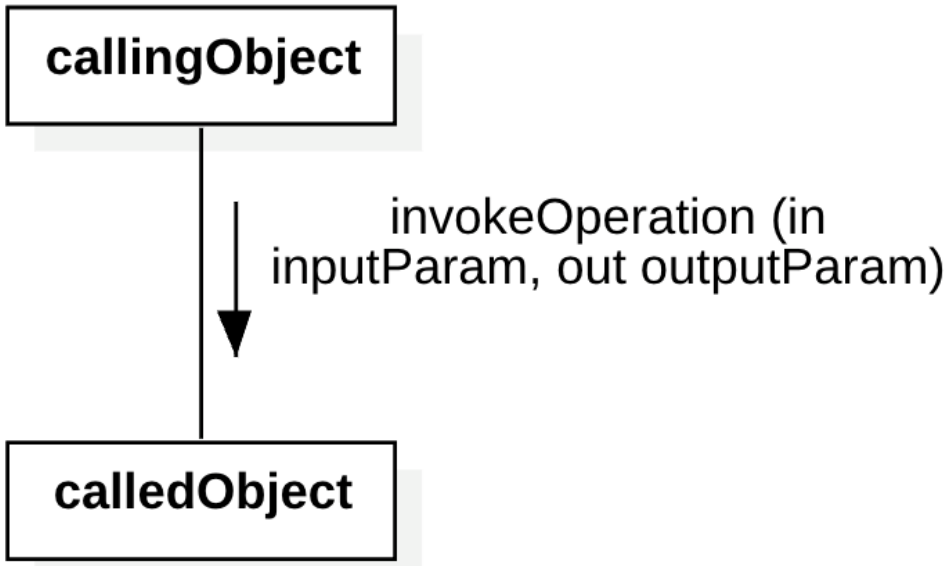


Figure 12.6. Example of layered architecture: Online Shopping System

CALL/RETURN PATTERN

- Basic communication in **sequential design**
- **Passive objects** invoke operations (methods) on other objects
- **Synchronous communication** (call -> return control/output)
- Example:
  - `WithdrawalTransactionManager` invokes debit operation on `CheckingAccount`
- UML notation: Arrow with black arrowhead



CALL/RETURN PATTERN

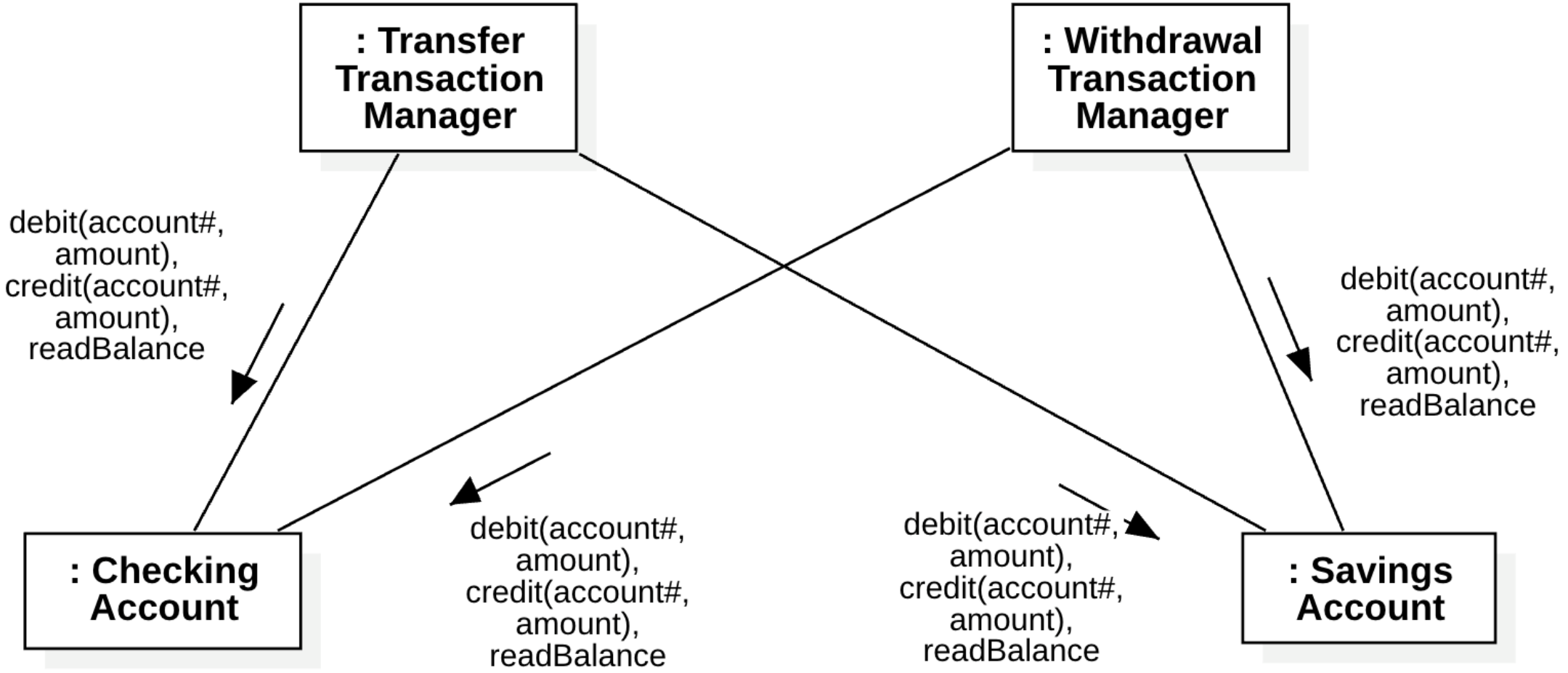


Figure 12.7. Call/return pattern

ASYNCHRONOUS MESSAGE COMMUNICATION PATTERN

- **Producer** sends message to **consumer** and **continues immediately**
- No waiting for reply; messages can queue (**FIFO** queue)
- Useful in **distributed/concurrent systems**
- Example
  - Automated Guided Vehicle System: Supervisory System Proxy sends async messages to Vehicle Control
- **Peer-to-peer** asynchronous communication also possible
- UML notation: **Arrow with stick arrowhead**

ASYNCHRONOUS MESSAGE COMMUNICATION PATTERN

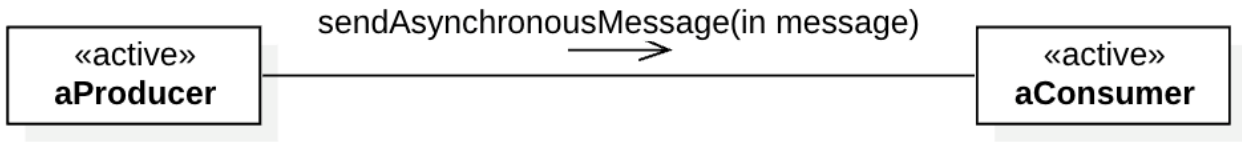


Figure 12.8. Asynchronous Message Communication pattern

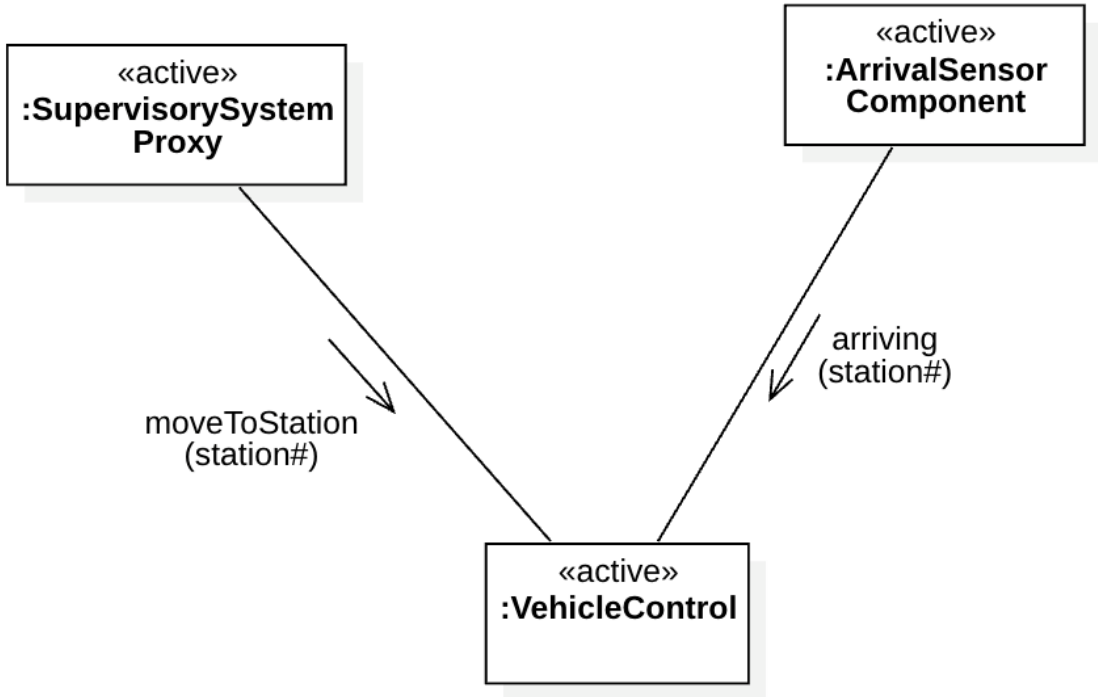


Figure 12.9. Example of the Asynchronous Message Communication pattern: Automated Guided Vehicle System

ASYNCHRONOUS MESSAGE COMMUNICATION PATTERN

- **Peer-to-peer** asynchronous communication. Two components send asynchronous messages to each other, referred to as **bidirectional asynchronous communication.**

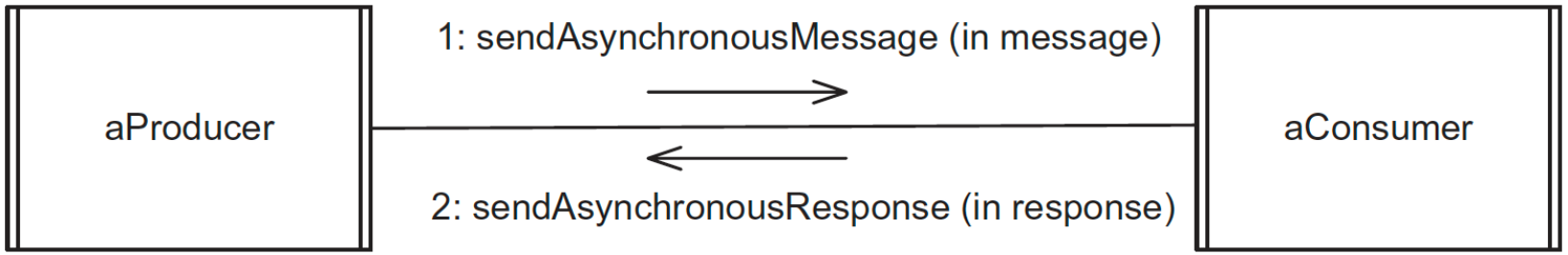


Figure 12.10. Bidirectional Asynchronous Message Communication pattern

SYNCHRONOUS MESSAGE COMMUNICATION WITH REPLY PATTERN

- Client sends request and **waits** for reply from service
- Used heavily in **client/server architectures**
- Service suspends if no message is available
- Example:
  - ATM Client sends ATMTransaction message to Banking Service, waits for bankResponse
- UML notation:
  - Black arrowhead for request

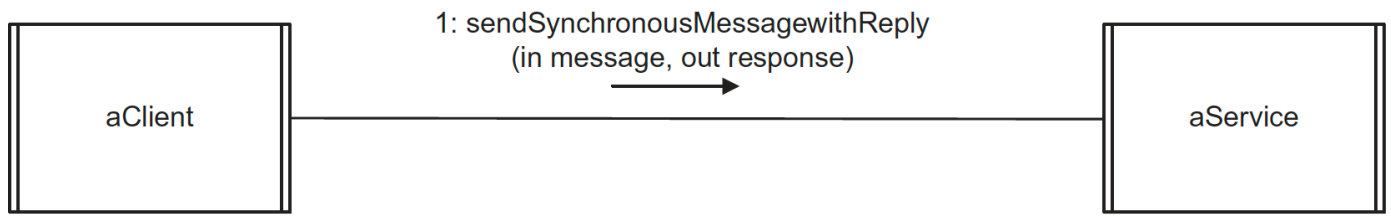


Figure 12.11. Synchronous Message Communication with Reply pattern

CONTENT

- Software Architecture and Component-based Software Architecture
- Multiple Views of a Software Architecture
- Software Architectural Pattern
- Documenting Software Architectural Patterns
- Interface Design
- Designing Software Architecture

DOCUMENTING SOFTWARE ARCHITECTURAL PATTERNS

A template for describing a pattern usually also addresses its strengths, weaknesses, and related patterns.

- **Pattern name**
- **Aliases**. Other names by which this pattern is known.
- **Context**. The situation that gives rise to this problem.
- **Problem**. Brief description of the problem.
- **Summary** of solution. Brief description of the solution.
- **Strengths of solution**
- **Weaknesses of solution**
- **Applicability**. When you can use the pattern.
- **Related patterns**
- **Reference**. Where you can find more information about the pattern.

DOCUMENTING SOFTWARE ARCHITECTURAL PATTERNS

Pattern name	Layers of Abstraction
Aliases	Hierarchical Layers, Levels of Abstraction
Context	Software architectural design
Problem	A software architecture that encourages design for ease of extension and contraction is needed.
Summary of solution	Components at lower layers provide services for components at higher layers. Components may use only services provided by components at lower layers.
Strengths of solution	Promotes extension and contraction of software design
Weaknesses of solution	Could lead to inefficiency if too many layers need to be traversed
Applicability	Operating systems, communication protocols, software product lines
Related patterns	Kernel can be lowest layer of Layers of Abstraction architecture. Variations of this pattern include Flexible Layers of Abstraction.
Reference	Chapter 12, Section 12.3.1; Hoffman and Weiss 2001; Parnas 1979.

CONTENT

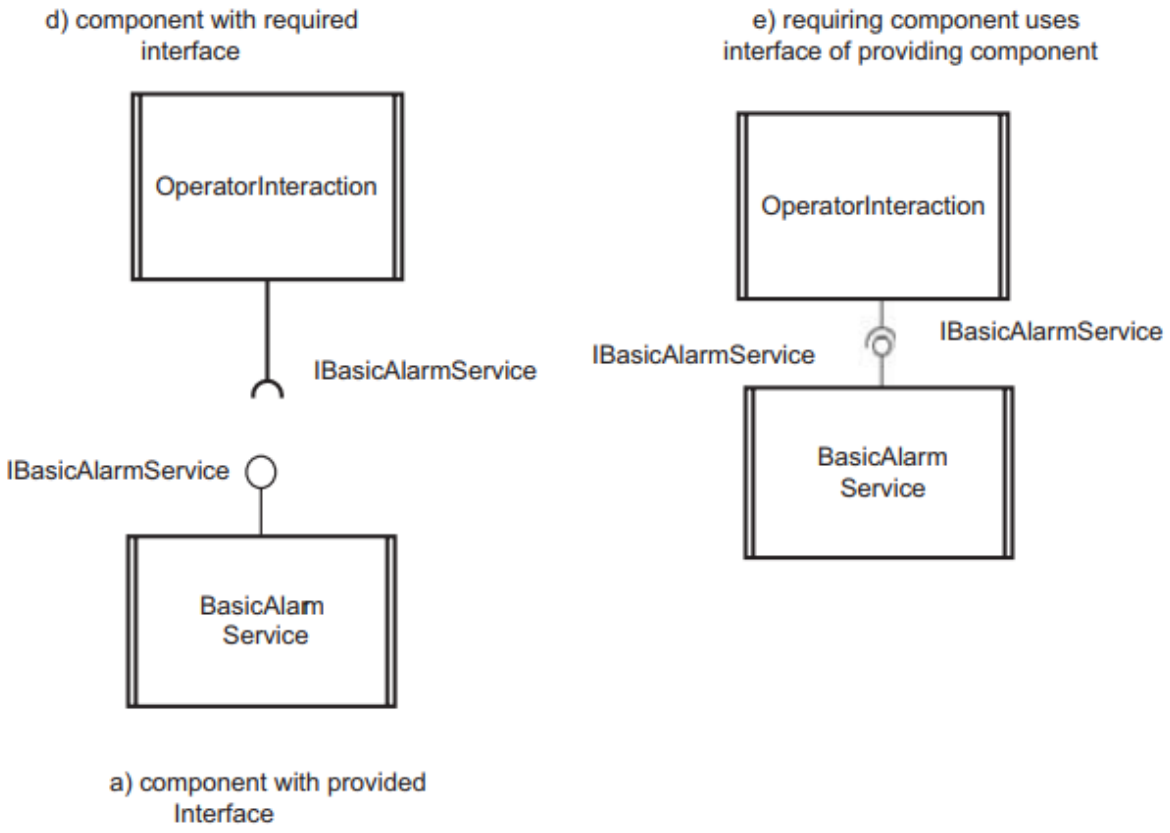
- Software Architecture and Component-based Software Architecture
- Multiple Views of a Software Architecture
- Software Architectural Pattern
- Documenting Software Architectural Patterns
- Interface Design
- Designing Software Architecture

INTERFACE DESIGN

- An important goal of both object-oriented design and component-based software architecture is the **separation of the interface from the implementation**.
- An **interface** specifies the externally visible operations of a class, service, or component without revealing the internal structure (implementation) of the operations
- The interface can be considered a *contract* between the designer of the external view of the class and the implementer of the class internals.
- It is also a *contract* between a class that requires (uses) the interface (i.e., invokes the operations provided by the interface) and the class that provides the interface

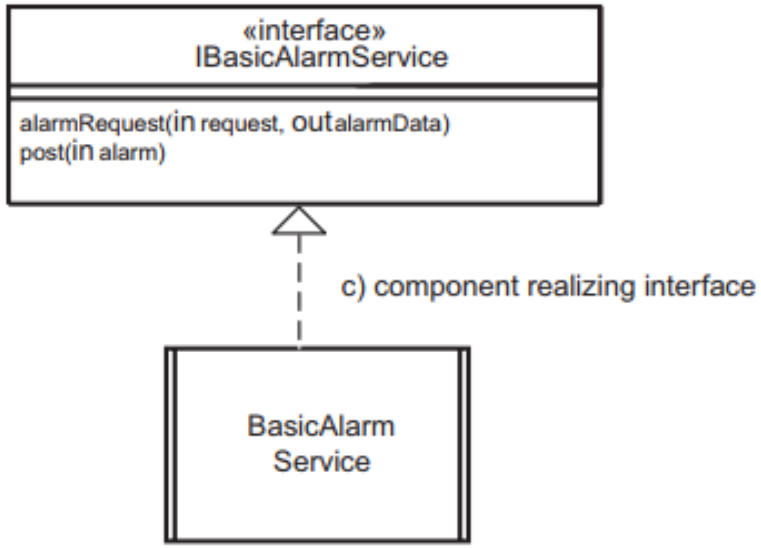
SYNCHRONOUS MESSAGE COMMUNICATION WITH REPLY PATTERN

- Interfaces for subsystems, distributed components, and passive classes can all be depicted using the same interface notation.
- By convention, the name starts with the letter "I."
- An interface can be depicted in two ways:
  - **Simple** (the interface is depicted as a little circle with the interface name next to it). The class or component that provides the interface is connected to the small circle



SYNCHRONOUS MESSAGE COMMUNICATION WITH REPLY PATTERN

b) specification of interface



Interface: IBasicAlarmService

Operations provided:

- alarmRequest (**in** request, **out** alarmData)
- post (**in** alarm)

6. DESIGNING SOFTWARE ARCHITECTURES

- During software design modeling, design decisions are made relating to the characteristics of the software architecture.

THE DESIGN OF DIFFERENT KINDS OF SOFTWARE ARCHITECTURES

- **Object-oriented software architectures.** Chapter 14 describes object-oriented design using the concepts of information hiding, classes, and inheritance.
- **Client/server software architectures.** Chapter 15 describes the design of client/ server software architectures.
- **Service-oriented architectures.** Chapter 16 describes the design of service-oriented architectures.
- **Distributed component-based software architectures.** Chapter 17 describes the design of component-based software architectures.
- **Concurrent and real-time software architectures.** Chapter 18 describes the design of real-time software architectures.
- **Software product line architectures.** Chapter 19 describes the design of software product line architectures
- Chapter 20 describes the quality attributes of software architectures that address nonfunctional requirements of software

SUMMARY

- This chapter presented an overview of software architecture.
- It described the multiple views of a software architecture: the static, dynamic, and deployment views.